

Plano — Distribuição automatizada do SmartEvents com eventos selecionados

Data: 2026-08-31

Tipo: feature

Objetivo: permitir selecionar um ou mais eventos e gerar, sem montagem manual, um pacote de eventos compacto ou um instalador Inno Setup completo contendo todas as configurações necessárias para esses eventos. A geração é uma operação de **quem distribui** e fica numa ferramenta interna do repositório (decisão 8); o aplicativo entregue ao usuário final apenas **importa** o resultado.

Contexto

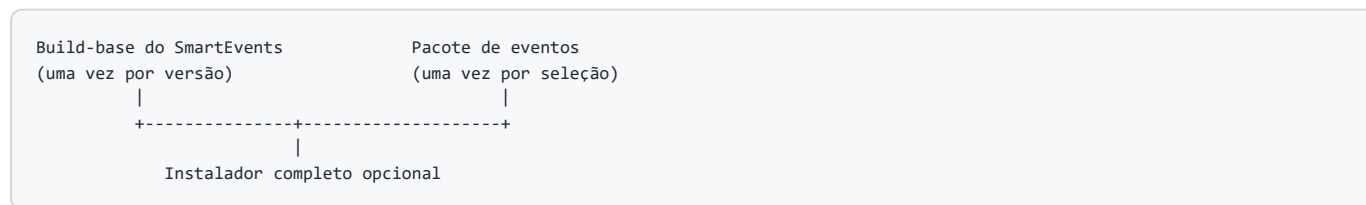
O projeto já possui quase todas as peças de um pipeline de distribuição:

- `build.py` recebe um perfil JSON, prepara uma semente, executa o PyInstaller, compila o Inno Setup, roda o self-test e gera hashes e manifestos;
- `tools/prepare_installer_seed.py` seleciona eventos, cliente, VIPs e logos;
- `main.spec` incorpora a semente e o catálogo de clientes dentro do bundle do PyInstaller;
- `installer/SmartEvents.iss` instala pré-requisitos e executa o diagnóstico final;
- o Smart Events Central já lista e mantém eventos, clientes, regionais, logos e VIPs em `server_data/`.

O problema é o acoplamento entre **versão do programa** e **conteúdo dos eventos**. Hoje, mudar somente a lista de eventos exige executar novamente o PyInstaller e recompilar um instalador de aproximadamente 579 MiB. O bundle ONEDIR instalado ocupa aproximadamente 883 MiB, enquanto os arquivos JSON dos eventos normalmente ocupam apenas alguns megabytes.

Além do custo de build, o fluxo atual exige criar manualmente um perfil em `build_profiles/` com `event_ids`, nomes de artefato, `AppId`, diretório de dados e README. Isso não escala quando a distribuição passa a ser uma operação frequente.

Este plano separa os dois ciclos:



O resultado final oferece duas saídas:

1. **Pacote `.sepack`** : pequeno, para importar eventos em uma instalação já existente.
2. **`Setup.exe` completo**: contém o build-base e o `.sepack`, para o operador receber um único arquivo e instalar tudo de uma vez.

Por que a implementação deve ser dividida em fases

A divisão em fases é necessária por dependência técnica, não apenas por tamanho:

```
Fase 1 – contrato .sepack, validação e importação segura
|
+--> Fase 2 – seleção múltipla e geração pela interface interna (estúdio)
      |
      +--> Fase 3 – build-base e Setup.exe completo sem novo PyInstaller
            |
            +--> Fase 4 – assinatura, homologação e retirada do fluxo legado
```

- A interface não pode oferecer a geração antes de existir um formato estável e importável.
- A geração nunca entra na interface do produto: ela vive no estúdio interno (decisão 8).
- O instalador completo só pode reutilizar o programa depois que houver um build-base sem evento embutido.
- Assinatura e homologação dependem do formato e do pipeline já estabilizados.
- O fluxo atual por perfil permanece disponível como fallback até o encerramento da Fase 4.

Cada fase produz algo utilizável e pode ser validada isoladamente.

Decisões de arquitetura que valem para todas as fases

1. Um único SmartEvents, vários eventos

O produto passa a usar um `AppId` estável e o diretório persistente `%LOCALAPPDATA%\SmartEvents`. Eventos diferentes não criam instalações diferentes. Um `AppId` exclusivo continua possível somente para uma distribuição que precise, por decisão explícita, funcionar lado a lado e sem compartilhar nenhum dado.

No Inno Setup, instalações com o mesmo `AppId` são tratadas como a mesma aplicação e podem continuar o mesmo registro de instalação/desinstalação. Referência: [AppId](#) e [Same Application](#).

2. O pacote nunca contém segredos ou histórico

O gerador deve bloquear a inclusão de:

- `credentials.json`;
- `session.json`, cookies ou tokens;
- bancos `smart_events*.db`;
- logs e diagnósticos locais;
- resultados de coleta, alertas e relatórios gerados;
- caminhos locais do operador que não façam parte da configuração funcional do evento.

O pacote pode conter endereço do OSS, cliente, regional, IDs de tasks e dados de VIPs. Como isso também pode ser sensível, a tela de revisão precisa mostrar exatamente o que será incluído.

3. Pacotes aceitam vários clientes

O formato novo não repete a limitação atual de exatamente um cliente. A seleção pode conter eventos de clientes distintos, desde que cada evento encontre o cadastro do seu cliente e da sua regional. As credenciais continuam sendo pedidas localmente no primeiro uso e nunca são transportadas.

4. Inclusão mínima de VIPs

Para cada evento:

- se `event.vips` declarar associações, incluir somente os VIPs referenciados;
- para evento legado sem associações explícitas, detectar os VIPs compatíveis por cliente + regional, mostrar essa seleção como **fallback de compatibilidade** e exigir que o operador a revise antes de gerar;
- permitir desmarcar VIPs, mas avisar quando isso remover uma associação utilizada pelo evento;
- nunca incluir automaticamente todos os VIPs de um cliente sem considerar a regional.

5. Importação incremental e preservação por padrão

O pacote não é uma nova fonte autoritativa para todo o diretório `server_data`. Ele só altera os itens que carrega. Portanto:

- evento ausente: adicionar;
- mesmo ID e mesmo hash: tratar como operação idempotente;
- mesmo ID e conteúdo diferente: preservar o local por padrão e registrar conflito;
- substituição: somente com opção explícita e backup do arquivo anterior;
- eventos que não aparecem no pacote: nunca apagar;
- clientes, regionais e VIPs existentes: conciliar por ID conforme a decisão 5.1, nunca duplicar;
- uma falha de validação: não gravar nenhum arquivo.

Essa regra precisa ser separada de `sync_events_from_server()`, que hoje remove eventos locais ausentes na resposta do servidor. Importação de pacote é uma operação incremental; sincronização autoritativa continua sendo outro modo.

5.1 Conciliação de clientes, regionais e VIPs

O evento é a unidade funcional do pacote e continua sujeito à regra acima: mesmo ID com conteúdo diferente é preservado inteiro e registra conflito. Cliente, regional, VIP e logo são **cadastros compartilhados entre eventos** e recebem tratamento diferente: são conciliados campo a campo e nunca duplicados. Importar um evento novo da TIM em uma instalação que já tem a TIM cadastrada não pode criar um segundo cliente TIM.

Identidade:

- a chave de conciliação é o `id` do cadastro, nunca o nome do arquivo nem o nome de exibição;
- regional é conciliada pelo par cliente + `region` ;
- se o pacote trazer um `id` novo cujo nome normalizado já existe localmente sob outro `id` , não criar nem mesclar automaticamente: registrar aviso na revisão e no relatório para decisão do operador.

Regras de campo, válidas para cliente e VIP:

- campo ausente localmente e presente no pacote: adicionar;
- campo presente localmente e ausente, nulo ou vazio no pacote: manter o valor local;
- mesmo campo preenchido nos dois lados com valores diferentes: preservar o local por padrão e registrar conflito, igual à política de evento;
- `regionais` : adicionar as regiões que faltam, manter as regiões locais ausentes no pacote e, para a mesma região com `ip` divergente, preservar o local e registrar conflito;
- nenhum campo local é apagado por ausência no pacote.

Exceção deliberada para logo/imagem:

- logo ausente ou vazio no pacote: manter o logo local;
- logo presente no pacote e diferente do local: atualizar, gravando backup do arquivo anterior e registrando a substituição no relatório;
- essa é a única substituição automática sem opção explícita, porque o logo é recurso de apresentação e não altera o comportamento da coleta.

Resultado esperado:

- cadastro que só recebeu campos faltantes é reportado como `reconciled` , e não como `added` ou `replaced` ;
- conciliação sem nenhuma diferença é idempotente e não regrava o arquivo;
- a conciliação acontece em memória, sobre o estado local validado, antes de qualquer gravação; uma falha posterior faz rollback junto com o restante da importação.

6. Geração restrita e sem comandos arbitrários

Os endpoints de geração (do estúdio da decisão 8) recebem IDs de eventos e opções enumeradas. Eles não recebem comandos de shell, caminho de saída livre, caminho de `.iss` ou argumentos de PyInstaller/Inno Setup. Toda saída fica dentro de um diretório controlado pelo aplicativo.

7. Um build-base por versão

O PyInstaller só deve rodar quando o código, uma dependência ou um recurso do programa mudar. A seleção de eventos nunca deve dispará-lo. O manifesto do build-base é a chave do cache e inclui:

- versão do SmartEvents;
- commit e indicação de worktree suja;
- versão do Python e arquitetura;
- hashes do executável, bundle e dependências;
- revisão dos navegadores Playwright;
- versão do schema `.sepack` suportada.

8. A geração fica no repositório; o produto só recebe o resultado

O SmartEvents distribuído ao usuário final **não gera distribuições**. Selecionar eventos, revisar dependências e produzir `.sepack / Setup.exe` é operação de quem distribui, e vive numa ferramenta separada do repositório — o **Distribution Studio** (`tools/distribution_studio.py` + `tools/distribution_studio.html`).

O que é distribuído são os **resultados** deste plano; não a ferramenta que os produz.

Consequências que valem para as Fases 2, 3 e 4:

- o `server.py` da Central não expõe nenhum endpoint de distribuição e o `server_frontend/index.html` não tem seleção múltipla, botão de gerar nem link para o estúdio: a interface do produto é exatamente a que era antes desta feature;
- o estúdio roda em outro processo e em outra porta (`python -m tools.distribution_studio`), sob o prefixo `/distribution-studio` . Não existe rota em `/` : sem o link, não há tela;
- ele escuta apenas em `127.0.0.1` e não habilita CORS. A Central serve a rede local; o estúdio não sai da máquina de quem gera;
- nem `server.py` nem `main.spec` importam `core/distribution_service.py` , então o serviço de geração **não entra no bundle do PyInstaller**. O importador (`core/event_package.py`) continua embarcado — quem recebe o `.sepack` precisa dele;
- o estúdio lê o mesmo `server_data` da Central (`core.paths.server_data_dir()`) e é somente-leitura sobre ele: cadastrar, editar e excluir evento continua sendo função exclusiva da Central.

Esconder a tela não é o controle de segurança: as regras das decisões 2 e 6 (nada de segredo no pacote, nenhum caminho ou comando vindo do navegador) continuam valendo integralmente dentro do estúdio, e são elas que os testes travam.

Contrato proposto para o arquivo `.sepack`

`.sepack` é um ZIP com extensão própria. Na Fase 1 ele já nasce como um envelope preparado para assinatura, evitando quebrar o formato na Fase 4:

```
SmartEvents_Eventos_<nome>_<data>.sepack
├─ envelope.json
├─ payload.zip
└─ payload.zip.issig          # opcional nas Fases 1-3; obrigatório em produção na Fase 4

payload.zip
├─ manifest.json
├─ events/<event-id>.json
├─ clientes/<cliente-id>.json
├─ vips/<vip-id>.json
└─ logos/<arquivo>
```

Campos mínimos de `envelope.json` :

```
{
  "schema_version": 1,
  "payload": "payload.zip",
  "signature": "payload.zip.issig",
  "signature_required": false
}
```

Campos mínimos de `manifest.json` :

```
{
  "schema_version": 1,
  "package_id": "uuid",
  "name": "Barretos 2026",
  "created_at_utc": "2026-08-31T12:00:00Z",
  "created_by_app_version": "1.0.0",
  "minimum_app_version": "1.0.0",
  "event_ids": ["festa-do-peao-de-barretos-2026"],
  "clients": ["Vivo"],
  "files": {
    "events/festa-do-peao-de-barretos-2026.json": "sha256..."
  },
  "contains_credentials": false
}
```

Regras do formato:

- nomes e IDs normalizados, sem caminhos absolutos ou `..` ;
 - hashes SHA-256 de todos os arquivos do payload;
 - JSON UTF-8, objetos no topo e tamanho máximo configurado;
 - ZIP sem links simbólicos e sem entradas duplicadas;
 - limite de quantidade, tamanho compactado e tamanho descompactado para evitar ZIP bomb;
 - compatibilidade decidida por `schema_version` e `minimum_app_version` , nunca pelo nome do arquivo;
 - ordem determinística de arquivos e timestamps normalizados quando possível, para permitir reproduzir o mesmo payload a partir da mesma fonte.
-

Fase 1 — Formato `.sepack`, gerador e importação segura

Explicação simples

Esta fase cria o “contêiner” compartilhável dos eventos e ensina o SmartEvents a validá-lo e importá-lo. Ao final, já será possível gerar e importar pacotes pela linha de comando, sem alterar a tela e sem gerar um novo executável.

O objetivo é estabilizar primeiro o contrato e as regras de preservação. A interface da Fase 2 será apenas uma forma amigável de chamar essas operações.

Escopo detalhado

Código

Novo `core/event_package.py`

- Definir constantes de schema, limites de tamanho e extensões permitidas.
- Criar dataclasses ou estruturas equivalentes para:
 - `PackageManifest` ;
 - `PackagePreview` ;
 - `ImportPlan` ;
 - `ImportResult` ;
 - `ImportConflict` ;
 - `RecordReconciliation` (cadastro, campos adicionados, campos preservados, conflitos).
- Implementar `preview_package(source_dir, event_ids, vip_policy)` :
 - resolver os eventos pelo `id` interno, e não apenas pelo nome do arquivo;
 - coletar clientes, regionais, logos, tasks PM, clusters e VIPs necessários;
 - identificar erros e avisos sem criar artefato;
 - impedir credenciais, sessões, banco ou logs no conjunto selecionado.
- Implementar `build_package(preview, destination)` :
 - montar `payload.zip` em staging temporário;
 - calcular hashes e escrever `manifest.json` por último;
 - montar o envelope `.sepack` por substituição atômica;
 - apagar staging mesmo em caso de erro;
 - recusar sobrescrever arquivo existente, salvo com `--force` explícito.
- Implementar `inspect_package(path)` e `validate_package(path)` :
 - verificar envelope, schema, compatibilidade, hashes e estrutura;
 - validar todos os JSONs com as regras já existentes em `core.seed` ;
 - validar referência evento → cliente → regional;
 - validar referência evento → VIP e logo;
 - retornar erros estruturados com `code` , `path` e mensagem para a UI.
- Implementar `reconcile_record(local, incoming, kind)` como função pura, sem I/O:
 - aplicar as regras de campo da decisão 5.1 e devolver o registro resultante mais a lista de campos adicionados, preservados e em conflito;
 - tratar `regionais` pela chave `region` e o logo pela exceção documentada;
 - ser a única origem da política de mesclagem, usada tanto pelo plano quanto pela importação.
- Implementar `plan_import(path, target, conflict_policy)` sem gravar nada, já apresentando o resultado da conciliação de cada cliente, regional e VIP.
- Implementar `import_package(path, target, conflict_policy)` :

- validar integralmente antes da primeira gravação;
- adquirir lock exclusivo da pasta de dados;
- fazer backup apenas dos arquivos que serão substituídos;
- usar escrita em arquivo temporário + `os.replace` ;
- gravar um relatório em `diagnostics/imports/<package-id>.json` ;
- reverter arquivos já trocados se uma etapa posterior falhar;
- retornar `added` , `unchanged` , `reconciled` , `preserved` , `replaced` , `conflicts` e `warnings` .

core/seed.py

- Extrair as validações genéricas de evento/cliente/VIP do recorte histórico RoadShow/TIM.
- Manter `validate_seed()` compatível com os perfis atuais.
- Adicionar uma validação de coleção que aceite múltiplos clientes e uma lista exata de eventos.
- Não usar `seed_operator_data()` para importar `.sepack` : a sementeira "copia se ausente" e a importação têm contratos diferentes.

Novo tools/event_package.py

- Oferecer comandos de desenvolvimento e automação:

```
python -m tools.event_package preview --events evento-a evento-b
python -m tools.event_package build --events evento-a evento-b --output outputs/distributions
python -m tools.event_package inspect pacote.sepack
python -m tools.event_package import pacote.sepack --data-dir .tmp/import-test
```

- Saída humana no terminal e opção `--json` para testes/pipeline.
- Códigos de saída distintos para argumento inválido, pacote inválido, conflito e erro interno.

main.py

- Tratar os modos sem GUI antes de importar/inicializar pywebview:

```
SmartEvents.exe --inspect-event-package <arquivo> --report <json>
SmartEvents.exe --import-event-package <arquivo> --conflict preserve --report <json>
SmartEvents.exe --import-event-package <arquivo> --show-dialog
```

- `--show-dialog` mostra apenas sucesso, aviso ou erro final; não abre a janela principal.
- Retornar código `0` somente quando a importação foi concluída ou era idempotente.
- Não iniciar servidor, scheduler ou navegador nesses modos.

core/paths.py

- Centralizar os caminhos de `distributions` , `diagnostics/imports` , backups e lock.
- Reutilizar a validação de nomes seguros já aplicada a `runtime_data_dir` .

`requirements-build.lock` e `main.spec`

- Nesta fase não retirar a semente embutida.
- Incluir os módulos novos automaticamente no bundle e confirmar que nenhum arquivo de teste, staging ou chave privada entrou no PyInstaller.
- Preparar a inclusão futura da chave pública/ `ISSigTool.exe` , sem exigir assinatura ainda.

Interface

Não haverá nova tela nesta fase. O único comportamento visível é o diálogo de resultado quando o executável for chamado com `--show-dialog`.

Mensagens esperadas:

- "Pacote importado: 2 eventos adicionados."
- "Pacote já aplicado; nenhum arquivo foi alterado."
- "O evento X já existe com alterações locais e foi preservado."
- "Cliente TIM já cadastrado: 1 regional adicionada, nenhum dado local alterado."
- "Pacote inválido: hash divergente em events/X.json."

Testes

Novo `tests/test_event_package.py`

teste	comportamento travado
<code>test_build_package_contains_selected_events_only</code>	evento não selecionado não entra
<code>test_package_includes_every_referenced_client_and_region</code>	dependências de cliente completas
<code>test_package_supports_events_from_multiple_clients</code>	fim da limitação de um cliente
<code>test_explicit_vip_references_win_over_legacy_fallback</code>	pacote mínimo e previsível
<code>test_legacy_vip_fallback_is_reported_for_review</code>	fallback nunca fica silencioso
<code>test_credentials_sessions_databases_and_logs_are_rejected</code>	segredo/histórico não entra
<code>test_manifest_hashes_cover_every_payload_file</code>	integridade do payload
<code>test_package_build_is_reproducible_for_same_inputs</code>	conteúdo determinístico
<code>test_validator_rejects_path_traversal</code>	bloqueio de <code>../</code> e caminho absoluto
<code>test_validator_rejects_duplicate_zip_entries</code>	uma entrada não mascara outra
<code>test_validator_rejects_zip_bomb_limits</code>	limite compactado/descompactado
<code>test_import_is_idempotent</code>	importar duas vezes não altera nada
<code>test_import_preserves_conflicting_local_event_by_default</code>	dado local não é perdido
<code>test_existing_client_is_reconciled_instead_of_duplicated</code>	evento novo do mesmo cliente não cria segundo cadastro
<code>test_reconcile_adds_missing_fields_and_keeps_local_values</code>	conciliação só acrescenta
<code>test_reconcile_merges_regions_and_preserves_divergent_ip</code>	regional nova entra, IP local sobrevive
<code>test_non_empty_logo_replaces_with_backup_and_empty_logo_is_ignored</code>	exceção do logo
<code>test_same_name_with_different_id_warns_instead_of_merging</code>	ambiguidade não resolvida em silêncio
<code>test_reconcile_without_differences_does_not_rewrite_file</code>	conciliação idempotente
<code>test_replace_creates_backup_before_overwrite</code>	substituição recuperável
<code>test_import_rolls_back_when_commit_fails</code>	sem importação parcial
<code>test_import_never_deletes_unrelated_events</code>	pacote é incremental
<code>test_newer_schema_or_minimum_version_is_rejected</code>	compatibilidade explícita

`tests/test_installer.py`

- Manter os testes do recorte legado.
- Acrescentar testes da validação genérica multi-evento/multicliente.
- Garantir que `credentials.json` continue ausente de qualquer artefato.

Novo `tests/test_main_event_package_cli.py`

- Verificar parsing, códigos de saída, relatório JSON e execução sem inicializar pywebview.

Como validar e testar

Testes automatizados

```
.venv-build\Scripts\python.exe -m pytest tests\test_event_package.py tests\test_installer.py tests\test_main_event_package_cli.py -q
.venv-build\Scripts\python.exe -m pytest tests -q --basetemp=.pytest-work\event-package-fase1
```

Critérios:

- zero falhas;
- a suíte existente não pode encolher;
- nenhum teste utiliza credencial real ou `server_data` mutável do workspace;
- testes de falha usam somente diretórios temporários.

Validação funcional e visual

1. Gerar um pacote com um evento TIM e outro Vivo.
2. Inspecionar o `.sepack` com uma ferramenta ZIP e confirmar a estrutura prevista.
3. Pesquisar por `password`, `cookie`, `session` e `credential` no conteúdo extraído; nenhum segredo deve existir.
4. Importar em `SMARTEVENTS_DATA_DIR` temporário e abrir o app apontando para essa pasta.
5. Confirmar que os dois eventos aparecem no seletor e que os dois clientes aparecem no gerenciador de credenciais.
6. Importar o mesmo pacote novamente: o diálogo deve informar operação idempotente.
7. Alterar localmente um dos eventos e importar novamente: o local deve ser preservado e o conflito deve aparecer no diálogo/relatório.
8. Importar um segundo pacote com outro evento do mesmo cliente: nenhum cliente novo deve ser criado; conferir no JSON local que regionais faltantes foram acrescentadas, que os campos locais preenchidos continuam intactos e que o relatório lista o cadastro como `reconciled`.
9. Repetir o passo anterior com o cliente local sem logo e o pacote com logo, depois com o cliente local com logo e o pacote sem logo: o logo deve entrar no primeiro caso e ser mantido no segundo, com backup no caso de substituição.
10. Corromper um byte de `payload.zip`: a importação deve parar antes de gravar qualquer arquivo.

Critério de aceite da fase

- Um pacote pode ser criado e importado sem executar PyInstaller ou Inno Setup.
- A importação não apaga eventos existentes e não transporta segredos.
- Clientes, regionais e VIPs já cadastrados são conciliados por ID, sem duplicar cadastro e sem perder dado local.
- O pacote é autodescritivo, verificável e compatível com assinatura posterior.

Sugestão de commit

```
feat: add safe event package generation and import
```

Fase 2 — Seleção múltipla e geração no Distribution Studio

Correção de rota (2026-09-01)

A primeira execução desta fase colocou a seleção múltipla e o fluxo de geração **dentro do Smart Events Central**. Isso estava errado: a Central é o produto que o usuário final recebe, e gerar artefato é operação de quem distribui. A fase foi refeita conforme a decisão 8.

O que mudou em relação à versão anterior deste plano:

- `server.py` e `server_frontend/index.html` voltaram exatamente ao estado anterior à fase — sem endpoints, sem checkbox, sem barra de seleção, sem modal e sem link;
- o mesmo fluxo (seleção → revisão → job → download) passou para o Distribution Studio, uma ferramenta interna servida em outro processo, sob `/distribution-studio`;
- `tests/test_distribution_api.py` virou `tests/test_distribution_studio_api.py` e `tests/test_server_frontend_distribution_ui.py` virou `tests/test_distribution_studio_ui.py`, com testes novos que travam a limpeza da Central e a ausência do estúdio no bundle.

A Fase 1 não foi afetada: o formato, o gerador, o validador e o importador continuam no `core/`, e a importação por linha de comando continua sendo função do executável distribuído.

Explicação simples

Esta fase transforma os comandos da Fase 1 em um fluxo visual **para quem distribui**. Quem gera abre `http://127.0.0.1:8010/distribution-studio`, seleciona eventos na lista, revisa dependências e avisos, escolhe “Pacote de eventos” e baixa o `.sepack` pronto.

O backend expõe operações específicas de distribuição e executa o trabalho em segundo plano para que a página não trave. A mesma infraestrutura de job será reutilizada pelo instalador completo na Fase 3.

Escopo detalhado

Código

Novo `core/distribution_service.py`

- Encapsular preview, criação de artefato, progresso e inventário de saídas.
- Manter estados: `queued`, `validating`, `packaging`, `compiling`, `testing`, `ready`, `failed`.
- Persistir status em `data/distributions/jobs/<job-id>.json`, permitindo recuperar o resultado depois de atualizar a página.
- Permitir um job de escrita por vez com lock; previews podem ocorrer em paralelo.
- Gerar artefatos somente em `data/distributions/<job-id>/`.
- Limpar jobs incompletos antigos por política configurável, sem apagar jobs `ready` automaticamente.
- Redigir logs antes de expô-los para a UI: nenhum segredo, variável de ambiente completa ou caminho de usuário desnecessário.
- Não pode ser importado por `server.py` nem declarado em `main.spec`: é o import que arrastaria a geração para dentro do executável distribuído.

Novo `tools/distribution_studio.py`

Aplicação FastAPI própria, com o seu próprio `uvicorn`:

```
python -m tools.distribution_studio          # http://127.0.0.1:8010/distribution-studio
python -m tools.distribution_studio --port 8020
```

Endpoints, todos sob o prefixo da ferramenta:

```
GET  /distribution-studio
GET  /distribution-studio/api/events
GET  /distribution-studio/api/capabilities
POST /distribution-studio/api/preview
POST /distribution-studio/api/jobs
GET  /distribution-studio/api/jobs/{job_id}
GET  /distribution-studio/api/jobs/{job_id}/download
GET  /distribution-studio/api/jobs/{job_id}/manifest
```

Regras:

- não existe rota em `/`, e nenhuma rota fora do prefixo: sem o link não há tela;
- bind fixo em `127.0.0.1` e nenhum middleware de CORS;
- aceitar somente IDs existentes e opções enumeradas;
- validar quantidade máxima de eventos;
- não aceitar caminho de origem/saída vindo do navegador — um campo `source` / `output` / `iss` extra no corpo é recusado com 400;
- validar `job_id` antes de compor qualquer caminho;
- usar `FileResponse` apenas para arquivo registrado no manifesto do job;
- bloquear download enquanto o job não estiver `ready`;
- nesta fase, `capabilities.formats` retorna apenas `event_package`;
- `GET .../api/events` devolve só o resumo que a lista desenha (id, nome, status, datas, cliente, regional, contagem de sites e células) — o evento inteiro não precisa atravessar a rede para o operador marcar uma caixa;
- a origem é sempre `core.paths.server_data_dir()`, a mesma pasta lida pela Central, e o estúdio nunca grava nela: não há POST/PUT/DELETE de cadastro.

Novo `tools/distribution_studio.html`

Página única e autocontida (CSS e JS inline, sem dependência externa), fora de `frontend/` e de `server_frontend/` — as duas pastas entram no bundle do PyInstaller.

- `<meta name="robots" content="noindex, nofollow">` e um aviso permanente de que a tela é interna e não é distribuída com o aplicativo.
- Lista de eventos somente-leitura, com checkbox acessível por card.
- “Selecionar todos visíveis”, contador de selecionados e botão “Gerar distribuição”.
- Nenhuma ação de editar/excluir: o estúdio não altera `server_data`.
- Modal de revisão com:
 - nome da distribuição e nome de arquivo sugerido;
 - eventos, clientes e regionais;
 - sites, células, clusters e tasks PM por evento;
 - VIPs explícitos e VIPs incluídos por fallback legado;
 - nota explícita de que cliente, regional, VIP e logo já existentes no destino serão conciliados por ID conforme a decisão 5.1, e não recriados nem sobrescritos;
 - avisos e erros de validação;
 - lista explícita “não será incluído”: credenciais, sessão, histórico e logs;
 - formato “Pacote de eventos (.sepack)”; o formato Setup aparece desabilitado com texto “disponível após configurar um build-base” até a Fase 3.
- O botão Gerar fica desabilitado com erro bloqueante e habilitado com aviso não bloqueante.
- Mostrar progresso por etapa usando polling com backoff.
- Em sucesso, mostrar tamanho, SHA-256, data e botões Baixar pacote/Baixar manifesto.
- Em falha, mostrar resumo, etapa que falhou e um bloco de diagnóstico copiável.
- Seleção deve sobreviver à re-renderização da lista e ser limpa ao concluir o download, conforme decisão de UX documentada no código.
- Toda requisição sai pela constante `STUDIO_API`; nenhuma URL é montada à mão.

`server.py` e `server_frontend/index.html`

Nada a fazer além de **não** ter nada: os dois permanecem no estado anterior a esta fase. Os testes travam isso explicitamente, porque `server` é `hiddenimport` do `main.spec` e qualquer endpoint adicionado aqui viajaria dentro do executável.

Interface

Fluxo esperado, na tela do estúdio:

```
Distribution Studio                      SmartEvents 1.0.0
Ferramenta interna – não distribuída com o aplicativo

[x] Selecionar todos visíveis   2 eventos selecionados   [Gerar distribuição]

[x] Evento A   Vivo • SP • 14 sites • 240 células       Ativo
[x] Evento B   TIM • RJ • 5 sites • 33 células          Ativo
[ ] Evento C   TIM • SP • 5 sites • 33 células          Agendado

  [ Revisar distribuição
  | 2 eventos • 2 clientes • 4 VIPs
  | ⚠ 2 VIPs vieram do fallback legado
  | Formato: Pacote .sepack
  | [Cancelar]                      [Gerar]
```


Estados visuais obrigatórios:

- nenhum evento selecionado;
- carregando preview;
- preview com erro;
- preview com aviso;
- job em progresso;
- sucesso com download;
- falha com possibilidade de tentar novamente.

Testes

Novo `tests/test_distribution_studio_api.py`

teste	comportamento travado
<code>test_event_list_summarizes_without_shipping_the_whole_event</code>	a lista expõe só o que desenha
<code>test_event_list_skips_unreadable_files_instead_of_failing</code>	um JSON quebrado não derruba a tela
<code>test_capabilities_lists_event_package_only_in_phase2</code>	capacidade real, sem opção falsa
<code>test_preview_returns_dependencies_counts_warnings_and_errors</code>	contrato da revisão
<code>test_preview_rejects_unknown_event_id</code>	entrada controlada
<code>test_create_job_uses_only_server_selected_paths</code>	navegador não injeta caminho
<code>test_job_transitions_to_ready_and_downloads_registered_file</code>	ciclo completo
<code>test_failed_job_keeps_redacted_diagnostic</code>	erro útil e seguro
<code>test_download_rejects_unready_or_unknown_job</code>	sem acesso indevido
<code>test_concurrent_writers_are_serialized</code>	dois artefatos não se misturam
<code>test_restart_recovers_completed_job_metadata</code>	atualização da página não perde saída
<code>test_central_server_exposes_no_distribution_endpoint</code>	a Central não gera nada
<code>test_central_frontend_has_no_selection_or_generation_ui</code>	a interface do produto voltou ao que era
<code>test_studio_never_enters_the_distributed_executable</code>	<code>main.spec</code> não empacota o estúdio
<code>test_studio_serves_nothing_outside_its_own_prefix</code>	sem o link, não há tela

Novo `tests/test_distribution_studio_ui.py`

- Testes estruturais de IDs, labels e funções JS do fluxo.
- Garantir que a página vive fora de `server_frontend/` e de `frontend/`.
- Garantir que a Central não tem link nem menção ao estúdio.
- Garantir que o estúdio é somente-leitura (sem `editEvent`, `deleteEvent`, `PUT` ou `DELETE`).
- Garantir que toda chamada sai pelo prefixo `/distribution-studio/api`.
- Garantir que erros bloqueiam Gerar e avisos não bloqueiam.
- Garantir que o formato Setup fica desabilitado sem capability.

Playwright

- Selecionar dois eventos, abrir preview e conferir nomes e contagens.
- Simular fallback de VIP e conferir o aviso.
- Executar job mockado, observar progresso e baixar o `.sepack`.
- Simular falha e conferir mensagem e botão Tentar novamente.
- Validar navegação por teclado, foco preso no modal e labels dos checkboxes.

Como validar e testar

Testes automatizados

```
.venv-build\Scripts\python.exe -m pytest tests\test_distribution_studio_api.py tests\test_distribution_studio_ui.py -q
.venv-build\Scripts\python.exe -m pytest tests -q --basetemp=.pytest-work\event-package-fase2
```

Validação visual

1. Abrir o Smart Events Central e confirmar que a interface é a de sempre: nenhum checkbox, nenhuma barra de seleção, nenhum botão de gerar e nenhum link para o estúdio.
2. Confirmar que `http://localhost:8000/api/distributions/capabilities` e `http://localhost:8000/distribution-studio` respondem 404 na Central.
3. Subir `python -m tools.distribution_studio` e confirmar que `http://127.0.0.1:8010/` responde 404 e que só o link completo abre a tela.
4. Selecionar um evento pelo checkbox e confirmar que nada é editado nem excluído.
5. Selecionar vários eventos, incluindo clientes diferentes.
6. Abrir a revisão e comparar todas as contagens com os cards da Central e com os JSONs de origem.
7. Conferir que credenciais/sessões/histórico aparecem explicitamente como excluídos.
8. Gerar o pacote e acompanhar os estados sem congelamento da página.
9. Baixar o `.sepack`, inspecioná-lo pela CLI da Fase 1 e importar em uma pasta limpa.
10. Recarregar o estúdio durante/depois do job e confirmar que o estado final pode ser recuperado.
11. Repetir em largura estreita: modal, cards e barra de seleção não devem cortar ações.
12. Navegar apenas com teclado e confirmar foco, `Esc`, seleção e geração.

Critério de aceite da fase

- Quem distribui consegue selecionar eventos e baixar um `.sepack` válido sem editar JSON ou executar comandos de empacotamento.
- A revisão mostra tudo que entra no pacote e tudo que fica de fora.
- O Smart Events Central continua idêntico ao que era antes desta feature: nenhuma rota, nenhum controle e nenhum link de geração.
- O serviço de geração não é alcançável a partir do executável distribuído.
- A interface não oferece Setup completo enquanto o host não possuir os requisitos da Fase 3.

Sugestão de commit

```
feat: generate event packages from an internal distribution studio
```

Fase 3 — Build-base reutilizável e instalador completo por seleção

Explicação simples

Esta fase elimina a recompilação do programa para cada evento. O PyInstaller gera um build-base sem eventos apenas uma vez por versão. Quando o operador escolhe "Instalador completo", o sistema combina esse build-base pronto com o `.sepack` e produz um novo `Setup.exe`.

O instalador copia o programa, importa o pacote na pasta de dados do usuário e só então executa o self-test. Para o destinatário, continua sendo um único arquivo e uma instalação comum.

Escopo detalhado

Código

```
main.spec
```

- Remover a dependência de `SMARTEVENTS_SERVER_DATA_SEED` do build-base novo.
- Não incorporar eventos, VIPs ou cadastro específico de cliente.
- Incorporar somente:
 - frontend e Central;
 - catálogos realmente globais do programa;
 - navegadores Playwright homologados;
 - perfil genérico de runtime com `runtime_data_dir=SmartEvents` ;
 - chave pública e verificador de pacote quando a Fase 4 os ativar.
- Manter temporariamente o caminho legado usado por `build.py --profile` , protegido por modo explícito, para permitir rollback durante a migração.

`build.py`

Reorganizar em subcomandos ou funções isoladas:

```
python build.py base
python build.py distribution --events evento-a evento-b --format package
python build.py distribution --events evento-a evento-b --format setup
python build.py legacy-profile --profile vivo-barretos-2026
```

- `base` :
 - validar Python/dependências/navegadores;
 - executar PyInstaller uma vez;
 - rodar self-test genérico;
 - gerar `dist/base/<version>/SmartEvents/` ;
 - gerar `SmartEvents-base.zip` e `base-manifest.json` ;
 - registrar tamanho descompactado para o Inno Setup;
 - recusar cache cujo manifesto/hash não corresponda ao conteúdo.
- `distribution --format package` delega ao serviço da Fase 1.
- `distribution --format setup` :
 - exigir build-base íntegro da mesma versão;
 - gerar `.sepack` em staging;
 - gerar wrapper `.iss` somente com definições controladas;
 - compilar o Setup sem chamar PyInstaller;
 - executar inspeção do instalador e smoke test;
 - gerar `build-manifest.json` , `SHA256SUMS.txt` , log e relatório final.
- Nenhum comando deve apagar `dist/base/<version>` ao gerar uma distribuição.
- A limpeza deve atuar apenas no diretório do job, sempre depois de validar o caminho resolvido.

Payload pré-comprimido do build-base

Para evitar recomprimir aproximadamente 883 MiB a cada seleção:

- gerar `SmartEvents-base.zip` uma vez no comando `base` ;
- armazená-lo no Setup com `nocompression` ;
- extraí-lo em `{tmp}` no início da instalação;
- usar o suporte de extração de arquivos do Inno Setup 7 para instalar em `{app}` e registrar os arquivos no log de desinstalação;
- usar `ArchiveExtraction=full` para ZIP;
- manter o pacote não sólido, conforme recomendação da documentação;
- verificar SHA-256/assinatura antes de extrair.

O fluxo deve ser prototipado e comparado com o `[Files] Source: "<base>\*"` atual. Se a estratégia de arquivo pré-comprimido não preservar corretamente atualização e desinstalação nos smoke tests, usar o fluxo recursivo seguro como fallback e registrar o custo de compressão; não liberar uma otimização que deixe arquivos órfãos. Referências: [Files/extractarchive](#) e [ArchiveExtraction](#).

`installer/SmartEvents.iss`

- Adicionar definições para `MyBaseArchive` , `MyBaseArchiveSize` , `MyEventPackage` e hashes.
- Usar o `AppId` estável do SmartEvents no modo genérico.
- Preservar `%LOCALAPPDATA%\SmartEvents` em atualização e desinstalação.
- Associar `.sepack` a:

```
SmartEvents.exe --import-event-package "%1" --show-dialog
```

usando `HKA\Software\Classes` , e remover apenas as chaves da associação no uninstall.

- Ordem de pós-instalação:
 1. instalar/atualizar o programa;
 2. validar o `.sepack` ;
 3. importar com `--conflict preserve` e gerar relatório;
 4. executar `--self-test` incluindo verificação dos IDs esperados;
 5. oferecer abertura do app somente se importação e self-test passarem.
- Reinício requerido por pré-requisito deve agendar **importação + self-test**, não apenas self-test.
- Falha de importação deve apresentar relatório e impedir mensagem de sucesso falso.
- Atualização do programa com pacote em conflito preservado pode concluir com aviso, mas precisa nomear os eventos não substituídos.

`core/self_test.py`

- Separar self-test genérico do programa e validação de pacote/distribuição.
- Build-base genérico não exige evento RoadShow ou cliente TIM.
- Aceitar `--expect-package-report` ou `--expect-events` para o smoke test do instalador.
- Confirmar que todos os eventos esperados aparecem no `server_data` do operador e no banco após sincronização local.

`core/seed.py` , `server.py` e `core/database.py`

- Inicialização genérica cria diretórios vazios válidos quando não há pacote.
- Primeiro boot sem evento deve abrir normalmente e mostrar estado vazio compreensível.
- Importação incremental não chama o caminho autoritativo que remove eventos ausentes.
- Após importação feita com o app aberto, sincronizar somente IDs afetados; na instalação, o próximo boot faz a sincronização completa local.

`core/distribution_service.py` e `tools/distribution_studio.py`

- `capabilities` passa a informar:

```
{
  "formats": ["event_package", "full_setup"],
  "base_version": "1.0.0",
  "base_ready": true,
  "iscc_ready": true
}
```

- Se cache, ISCC, pré-requisitos ou espaço em disco estiverem ausentes, `full_setup` não aparece e o motivo é mostrado.
- O job passa pelas etapas `packaging` , `compiling` , `testing` e `ready` .
- Impedir dois jobs de Setup simultâneos.

Interface

No modal do estúdio (Fase 2):

- habilitar “Instalador completo (.exe)” quando `capabilities` permitir;
- explicar:
 - `.sepack` : para quem já tem o SmartEvents;
 - `.exe` : primeira instalação ou atualização completa;
- mostrar versão e data do build-base usado;
- avisar que o Setup é grande mesmo sendo rápido de gerar;
- mostrar progresso separado de pacote, compilação e teste;
- em sucesso, disponibilizar Setup, manifesto e SHA-256;
- se o build-base estiver ausente/desatualizado, mostrar a ação administrativa necessária, sem tentar executar PyInstaller automaticamente por um clique comum da interface;
- o Setup completo continua sendo gerado apenas pelo estúdio: a Central não ganha nenhuma rota nova nesta fase.

Testes

`tests/test_build.py` ou novo `tests/test_distribution_build.py`

teste	comportamento travado
<code>test_base_build_has_no_event_or_client_specific_seed</code>	programa realmente genérico
<code>test_base_manifest_matches_every_cached_artifact</code>	cache confiável
<code>test_distribution_setup_never_invokes_pyinstaller</code>	ganho arquitetural principal
<code>test_distribution_rejects_missing_or_tampered_base</code>	não empacota cache inválido
<code>test_wrapper_uses_stable_app_id_and_data_dir</code>	atualização da mesma aplicação
<code>test_wrapper_imports_package_before_self_test</code>	ordem de instalação
<code>test_restart_path_replays_import_then_self_test</code>	pré-requisito com reboot
<code>test_sepack_file_association_is_scoped_and_uninstalled</code>	associação correta
<code>test_manifest_records_base_package_setup_and_exe_hashes</code>	rastreabilidade completa

`tests/test_installer.py`

- Build-base sem eventos passa no self-test genérico.
- Setup novo instala pacote com um e com vários eventos.
- Atualização da mesma versão é idempotente.
- Atualização de versão preserva credenciais, sessões, histórico e eventos não selecionados.
- Conflito de evento preserva local e produz aviso.
- Desinstalação remove programa/associação, mas preserva dados do operador.
- Reinstalação após uninstall encontra os dados preservados.

Testes de smoke do Inno Setup em diretório isolado

- instalação silenciosa com `/VERYSILENT /SUPPRESSMSGBOXES /NORESTART /LOG=...`;
- validação do relatório de importação e self-test;
- desinstalação silenciosa;
- comparação dos hashes dos arquivos instalados com o build-base;
- nenhuma dependência do diretório original do workspace após gerar o Setup.

Como validar e testar

Testes automatizados

```
.venv-build\Scripts\python.exe -m pytest tests\test_distribution_build.py tests\test_installer.py tests\test_distribution_api.py -q  
.venv-build\Scripts\python.exe -m pytest tests -q --basetemp=.pytest-work\event-package-fase3
```

Validação do pipeline

1. Executar `python build.py base` e registrar duração, tamanho e hashes.
2. Gerar três distribuições diferentes usando o mesmo base.
3. Confirmar pelos logs que o PyInstaller foi chamado somente no passo 1.
4. Confirmar que os três Setup usam o mesmo hash do executável/base e `.sepack` diferentes.
5. Comparar duração da geração por perfil atual com geração pelo base. A meta obrigatória é eliminar o PyInstaller; o tempo restante de Inno Setup deve ser registrado para decidir se o payload pré-comprimido fica habilitado.
6. Confirmar que o Setup é autocontido: copiar apenas o `.exe` para outra pasta e instalar.

Validação visual em máquina limpa

1. Abrir um Setup com dois eventos selecionados.
2. Confirmar nome, versão e publicador exibidos pelo instalador.
3. Concluir a instalação e observar a etapa "Importando eventos".
4. Abrir o SmartEvents e confirmar que somente os eventos do pacote aparecem numa pasta de dados realmente nova.
5. Ativar cada evento e confirmar mapa, sites, clusters, tasks e solicitação local de credenciais.
6. Gerar e abrir um `.sepack` por duplo clique; o diálogo deve importar sem reinstalar o programa.
7. Instalar outro Setup por cima e confirmar que os eventos anteriores não são apagados.
8. Alterar um evento local, instalar um pacote com o mesmo ID e confirmar preservação + aviso.
9. Desinstalar: programa e associação desaparecem; `%LOCALAPPDATA%\SmartEvents` permanece.

Matriz mínima de instalação nesta fase

cenário	resultado esperado
Windows 10 x64 limpo	instala, importa e abre
Windows 11 x64 limpo	instala, importa e abre
mesma versão novamente	idempotente
atualização de versão	preserva dados
pacote TIM + Vivo	ambos aparecem; credenciais continuam vazias
conflito local	preserva e informa
pacote corrompido	aborta antes do self-test
reboot por pré-requisito	retoma importação e diagnóstico

Critério de aceite da fase

- Gerar um Setup por seleção não executa PyInstaller.
- O destinatário recebe um único Setup autocontido.
- O mesmo SmartEvents acumula eventos de pacotes diferentes sem duplicar a instalação.
- Instalação, atualização, conflito, desinstalação e reinício têm smoke tests reproduzíveis.

Sugestão de commit

feat: build event installers from a reusable SmartEvents base

Fase 4 — Assinatura, auditoria, homologação e migração definitiva

Explicação simples

Esta fase transforma o fluxo funcional em um processo de distribuição confiável. Pacotes e instaladores passam a ter autenticidade verificável, o estúdio mantém histórico do que foi gerado e o fluxo antigo por perfil deixa de ser o caminho padrão somente depois da homologação.

Alguns itens dependem de infraestrutura externa, especialmente o certificado Authenticode. A ausência do certificado não bloqueia desenvolvimento, mas bloqueia classificar um artefato como “produção”.

Escopo detalhado

Código

Assinatura do `.sepack`

- Usar o `ISSigTool.exe` do Inno Setup para assinar `payload.zip` com ECDSA P-256.
- Guardar `payload.zip.issig` dentro do envelope `.sepack`, mantendo um único arquivo para envio.
- Incorporar no app somente a chave pública e o verificador; a chave privada:
 - fica fora do repositório;
 - é fornecida por caminho seguro no host de release;
 - nunca aparece em log, manifesto ou variável exibida na UI.
- Importador:
 - produção exige assinatura válida de uma chave permitida;
 - desenvolvimento pode aceitar pacote não assinado somente com flag explícita e aviso visível;
 - verificar assinatura antes de abrir/confiar no payload;
 - manter o arquivo verificado aberto ou trabalhar sobre cópia imutável para evitar TOCTOU.
- Permitir rotação de chave com lista de IDs públicos aceitos e data de desativação.

O Inno Setup documenta assinatura `.issig`, verificação por chave pública e a recomendação de manter o arquivo aberto durante o uso: [.issig](#), [ISSigKeys](#) e [ISSigVerify](#).

Assinatura Authenticode

- Configurar `SignTool` no Inno Setup para:
 - `SmartEvents.exe`;
 - `Setup.exe`;
 - uninstaller assinado.
- Usar SHA-256 e timestamp RFC 3161.
- Release de produção falha se assinatura estiver ausente ou inválida.
- Build de desenvolvimento deve declarar `signed=false` no manifesto, sem fingir confiança.
- Após assinar, validar a assinatura com ferramenta independente/Windows antes de publicar.

Referência: [SignTool](#).

Auditoria de distribuição

- Cada job pronto mantém:
 - usuário/host gerador, quando disponível sem coletar dado excessivo;
 - versão, commit e estado do source;
 - eventos, clientes e VIPs incluídos;
 - política de conflito;
 - hashes e status das assinaturas;
 - resultado dos testes/smoke;
 - timestamps de início/fim e duração por etapa.
- Criar endpoint/lista "Distribuições geradas" no estúdio, sob o mesmo prefixo.
- Permitir baixar novamente artefato e manifesto enquanto estiver dentro da retenção.
- Permitir excluir artefato individual com confirmação; manter manifesto de auditoria sem os dados do pacote, conforme política definida.

Migração e compatibilidade

- Marcar `build_profiles/` e `legacy-profile` como legado na documentação.
- Manter leitura dos instaladores/perfis existentes durante uma janela de transição.
- Não converter automaticamente instalações isoladas com `AppId` / `runtime_data_dir` próprios: documentar procedimento de exportar `.sepack`, instalar o SmartEvents genérico e importar.
- Só retirar a semente específica do caminho padrão depois que a matriz de homologação passar.
- Atualizar `README.md`, `installer/README-operador*.txt`, `docs/plano-inno-setup.md` e `docs/ORGANIZACAO.md`.

Interface

- Badge no resultado:
 - **Assinado e validado;**
 - **Não assinado — desenvolvimento;**
 - **Assinatura inválida** (sem download como produção).
- Nova seção "Distribuições geradas" **no estúdio**, com filtros por data, evento, cliente, formato e status. O histórico é registro de quem distribui e não aparece no produto.
- Exibir versão do build-base e validade da assinatura sem mostrar caminhos/chaves.
- Botão Excluir exige confirmação com nome do artefato e informa que o manifesto será preservado.
- Documentar na própria tela quando usar `.sepack` ou Setup completo.

Testes

`tests/test_event_package_signing.py`

teste	comportamento travado
test_signed_payload_is_accepted_with_allowed_public_key	caminho válido
test_tampered_payload_is_rejected_before_extraction	autenticidade antes do uso
test_signature_from_unknown_key_is_rejected	allowlist de chaves
test_missing_signature_requires_explicit_dev_mode	produção não aceita unsigned
test_private_key_path_and_contents_never_enter_artifacts_or_logs	segredo de release protegido
test_key_rotation_accepts_active_and_rejects_retired_key	operação futura

tests/test_distribution_audit.py

- Histórico persistente e filtrável.
- Manifesto de auditoria corresponde ao artefato.
- Exclusão remove binário, preserva registro e não permite path traversal.
- Logs e APIs não expõem chave, credencial ou ambiente completo.

Verificação de assinatura do Setup

- Teste/pipeline consulta assinatura de SmartEvents.exe , Setup e uninstaller.
- Certificado, cadeia, digest e timestamp precisam ser válidos.
- O teste de desenvolvimento aceita signed=false ; pipeline de release não aceita.

Homologação end-to-end

- Reexecutar toda a matriz da Fase 3 em VMs limpas.
- Transferir o Setup por ao menos um canal corporativo real e conferir hash depois da transferência.
- Validar instalação offline.
- Validar com antivírus/SmartScreen do ambiente homologado.
- Validar evento grande, múltiplos clientes, múltiplas regionais e pacote sem VIP.

Como validar e testar

Testes automatizados

```
.venv-build\Scripts\python.exe -m pytest tests\test_event_package_signing.py tests\test_distribution_audit.py tests\test_installer.py -c
.venv-build\Scripts\python.exe -m pytest tests -q --basetemp=.pytest-work\event-package-fase4
```

Validação de adulteração

1. Gerar pacote assinado e importar: sucesso.
2. Trocar um byte de `payload.zip` : assinatura inválida e zero arquivos gravados.
3. Substituir `.issig` por assinatura de outra chave: rejeição por chave desconhecida.
4. Remover a assinatura: rejeição em modo produção e aviso explícito em modo desenvolvimento.
5. Alterar um byte do Setup depois da assinatura: a verificação Authenticode deve falhar.

Validação visual e operacional

1. Gerar um `.sepack` e um Setup pelo estúdio; ambos aparecem no histórico.
2. Confirmar badge "Assinado e validado" e hashes iguais aos manifestos baixados.
3. Abrir propriedades do Setup no Windows e conferir a aba Assinaturas Digitais/publicador.
4. Instalar em VM limpa e confirmar que o UAC mostra o publicador esperado.
5. Importar pacote adulterado por duplo clique: diálogo claro, sem alteração nos eventos.
6. Excluir um artefato do histórico e confirmar que não pode mais ser baixado, mas a auditoria permanece.
7. Seguir apenas a documentação do operador em uma máquina sem o repositório e confirmar que o fluxo é suficiente.

Critério de aceite da fase

- Todo artefato de produção tem assinatura válida, hashes e manifesto de auditoria.
- Pacote adulterado ou de chave não confiável nunca é importado.
- A matriz Windows 10/11, instalação, atualização, conflito, reboot e uninstall está homologada.
- O fluxo por seleção passa a ser o caminho documentado; o legado fica apenas como fallback temporário e claramente identificado.

Sugestão de commit

```
feat: sign audit and homologate automated event distributions
```

Critérios de aceite globais

A implementação completa estará concluída quando:

- for possível selecionar um ou mais eventos no Distribution Studio;
 - a revisão resolver automaticamente clientes, regionais, logos, tasks e VIPs;
 - o usuário puder gerar `.sepack` ou Setup completo;
 - a seleção de eventos nunca executar PyInstaller;
 - o Setup completo for autocontido e usar build-base íntegro da versão escolhida;
 - importações forem atômicas, idempotentes e preservarem alterações locais por padrão;
 - eventos não presentes no pacote nunca forem apagados;
 - cadastros compartilhados (cliente, regional, VIP e logo) forem conciliados por ID, sem duplicar cadastro e sem apagar campo local;
 - credenciais, sessões, histórico, bancos e logs nunca entrarem no artefato;
 - pacote e Setup de produção tiverem autenticidade e integridade verificadas;
 - instalação/atualização preservarem `%LOCALAPPDATA%\SmartEvents` ;
 - o Smart Events Central e o executável distribuído continuarem sem qualquer recurso de geração — nem rota, nem controle, nem serviço empacotado;
 - os testes automatizados e a matriz visual/VM passarem sem regressão;
 - documentação de operador e desenvolvimento refletirem o novo fluxo.
-

Riscos e mitigação

risco	impacto	mitigação
Evento local ser sobrescrito	perda de configuração	preservar por padrão, preview, backup e rollback
Sincronização remover evento importado	perda após abrir o app	separar importação incremental da sincronização autoritativa
Cliente/VIP duplicado a cada pacote novo	cadastro fragmentado e credencial pedida de novo	conciliação por ID campo a campo, com relatório e aviso de nome ambíguo
Pacote carregar VIP indevido	exposição de dado	associação explícita; fallback legado visível e revisável
ZIP malicioso ou corrompido	gravação fora da pasta/DoS	path validation, limites, hashes e assinatura antes de extrair
Setup usar build-base errado	app/pacote incompatíveis	manifesto, hashes e <code>minimum_app_version</code>
Interface disparar comando arbitrário	execução de código	endpoints aceitam IDs/opções, nunca comando/caminho livre
Geração viajar dentro do app do usuário	superfície e operação indevidas no cliente	estúdio em processo/porta próprios, fora de <code>server.py</code> e do <code>main.spec</code> , com testes que travam a limpeza
Job concorrente misturar artefatos	Setup inconsistente	staging por UUID e lock de escrita/compilação
Otimização de archive deixar arquivos órfãos	uninstall incompleto	smoke de instalação/update/uninstall e fallback para <code>[Files]</code> recursivo
Certificado não disponível	release sem confiança	separar dev/release; produção falha sem assinatura
Migração de Applds isolados	dados dispersos	não migrar automaticamente; exportar/importar com roteiro explícito

Ordem recomendada de execução dentro de cada fase

Para reduzir retrabalho, cada fase deve seguir esta ordem:

1. fechar/atualizar o contrato de dados;
2. escrever primeiro os testes de segurança e preservação que poderiam causar perda de dados;
3. implementar backend/CLI;
4. implementar interface;
5. executar testes direcionados;
6. executar suíte completa;
7. fazer validação visual/funcional;
8. gerar evidências e atualizar documentação;
9. criar o commit sugerido somente com a fase aprovada.

Não iniciar a fase seguinte com testes vermelhos ou critério de aceite pendente na fase anterior.

Entregáveis finais

- `core/event_package.py` e testes de segurança/importação;
- CLI de geração, inspeção e importação;
- Distribution Studio interno: seleção múltipla, revisão e download, fora do produto;
- serviço e APIs de jobs de distribuição;
- `.sepack` importável e associado ao Windows;
- build-base genérico e cacheado;
- Setup Inno completo gerado sem novo PyInstaller;
- manifestos, SHA-256, assinatura e histórico de auditoria;
- smoke tests de instalação, atualização, conflito, desinstalação e reboot;
- documentação de desenvolvimento e de operador atualizada.